

BÀI 2: CÁC THUẬT TOÁN TÌM KIẾM: BFS, DFS VÀ UCS(tiếp theo)

I. MỤC TIÊU:

Sau khi thực hành xong, sinh viên nắm được:

- Thuật toán BFS, DFS trên cây tìm kiếm.
- Cài đặt được các thuật toán này trên máy tính.

II. TÓM TẮT LÝ THUYẾT:

1. Cấu trúc dữ liệu của các node trong cây tìm kiếm:

- State: trạng thái trong không gian trạng thái.
- Node: chứa 1 trạng thái, con trỏ tới predecessor, độ sâu, và chi phí đường đi, hành động.
- Depth: số bước dọc theo đường đi từ trạng thái ban đầu.
- Path Cost: chi phí đường đi từ trạng thái ban đầu tới node.
- Fringe: bộ nhớ lưu trữ các node mở rộng. Ví dụ, s là stack hoặc hàng đợi.

2. Các hàm thực thi:

- Make-Node(state): khởi tạo 1 node từ 1 trạng thái (state).
- Goal-Test(state): trả về true nếu state là trạng thái kết thúc.
- Successor-Fn(state): thực thi các hàm successor (mở rộng một tập các node mới với tất cả các hành động có thể áp dụng trong trạng thái).
- Cost(state, action): trả về chi phí thực thi hành động trong trạng thái.
- Insert(node, fringe): thêm 1 node mới vào fringe.
- Remove-First(fringe): trả về node đầu tiên từ fringe.

3. Tìm kiếm trên cấu trúc cây:

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure
fringe ← INSERT(MAKE-NODE(INITIAL-STATE[ problem ]), fringe)
loop do
  if EMPTY?(fringe) then return failure
  node ← REMOVE-FIRST(fringe)
  if GOAL-TEST[ problem ] applied to STATE[node] succeeds
    then return SOLUTION(node)
  fringe ← INSERT-ALL(EXPAND(node, problem), fringe)



---


function EXPAND(node, problem) returns a set of nodes

  successors ← the empty set
  for each ( action, result ) in SUCCESSOR-FN[ problem ](STATE[node]) do
    s ← a new NODE
    STATE[s] ← result
    PARENT-NODE[s] ← node
    ACTION[s] ← action
    PATH-COST[s] ← PATH-COST[node] + STEP-COST(node, action, s)
    DEPTH[s] ← DEPTH[node] + 1
    add s to successors
  return successors
```

Make-Node {

4. Thuật toán BFS:

5. Thuật toán DFS:

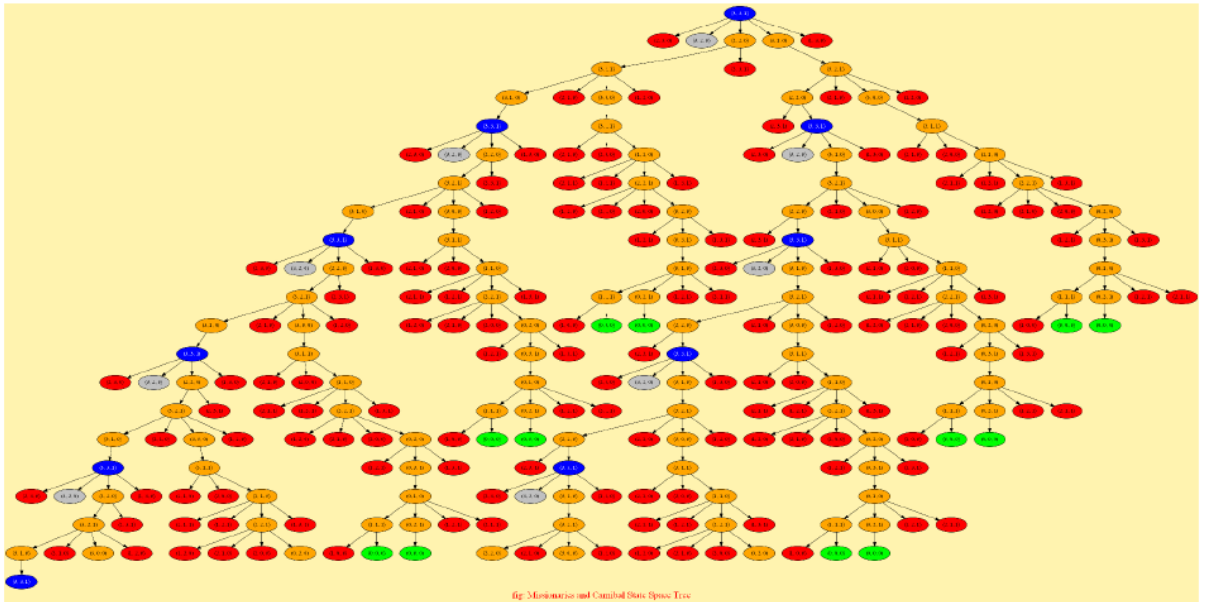
III. NỘI DUNG THỰC HÀNH:

1. Bài toán:

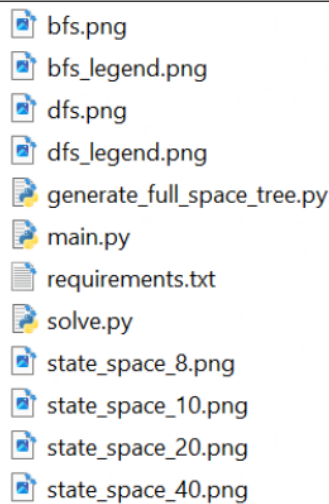
Có 3 người truyền giáo và 3 con quỷ ở bờ bên trái của một con sông, cùng với con thuyền có thể chở được 1 hoặc 2 người. Nếu số quỷ nhiều hơn số người truyền giáo trong một bờ thì số quỷ sẽ ăn thịt số người truyền giáo. Tìm các để đưa tất cả qua bờ sông bên kia (bên phải) sao cho số người không ít hơn số quỷ ở cùng 1 bờ (bên trái hay bên phải), nghĩa là không ai bị ăn thịt. Gọi (a, b, k) với $0 \leq a, b \leq 3$, trong đó a là số người, b là số con quỷ ở bên bờ bên trái, $k = 1$ nếu thuyền ở bờ bên trái và $k = 0$ nếu thuyền ở bờ bên phải. Khi đó, không gian trạng thái của bài toán được xác định như sau:

- Trạng thái ban đầu là $(3, 3, 1)$.
- Thuyền chở qua sông 1 người, hoặc 1 con quỷ, hoặc 1 người và 1 con quỷ, hoặc 2 người, hoặc 2 con quỷ $\Rightarrow$ các phép toán chuyển từ trạng thái này sang trạng thái khác là: $(1, 0)$, $(0, 1)$, $(1, 1)$, $(2, 0)$, $(0, 2)$ (trong đó (x, y) là số người và số quỷ di chuyển từ bờ bên trái qua bờ bên phải hay ngược lại).
- Trạng thái kết thúc là $(0, 0, 0)$.

2. Cây tìm kiếm(state_space_20.png):



3. Cài đặt:



A file explorer window showing the following files and folders:

- bfs.png
- bfs_legend.png
- dfs.png
- dfs_legend.png
- generate_full_space_tree.py
- main.py
- requirements.txt
- solve.py
- state_space_8.png
- state_space_10.png
- state_space_20.png
- state_space_40.png

```
generate_full_space_tree.py
6 from collections import deque
7 import pydot
8 import argparse
9 import os
10
11 # Set it to bin folder of graphviz
12 os.environ["PATH"] += os.pathsep + 'C:/Program Files/Graphviz/bin'
13
14 options = [(1, 0), (0, 1), (1, 1), (0, 2), (2, 0)]
15 Parent = dict()
16 graph = pydot.Dot(graph_type='graph',strict=False, bgcolor="#fff3af",
17                  label="fig: Missionaries and Cannibal State Space Tree",
18                  fontcolor="red", fontsize="24", overlap="true")
19 # To track node
20 i = 0
21
22 arg = argparse.ArgumentParser()
23 arg.add_argument("-d", "--depth", required=False,
24                 help="Maximum depth upto which you want to generate Space State Tree")
25
26 args = vars(arg.parse_args())
27
28 max_depth = int(args.get("depth", 20))
29
30 def is_valid_move(number_missionaries, number_cannibals):
31     """
32     Checks if number constraints are satisfied
33     """
34     return (0 <= number_missionaries <= 3) and (0 <= number_cannibals <= 3)
35
36 def write_image(file_name="state_space"):
37     try:
38         graph.write_png(f"{file_name}_{max_depth}.png")
39     except Exception as e:
40         print("Error while writing file", e)
41     print(f"File {file_name}_{max_depth}.png successfully written.")
42
43 def draw_edge(number_missionaries, number_cannibals, side, depth_level, node_num):
44     u, v = None, None
45     if Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)] is not None:
46         u = pydot.Node(str(Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)]),
47                        label=str(Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)][3]))
48         graph.add_node(u)
```

```

50     v = pydot.Node(str((number_missionaries, number_cannibals, side, depth_level, node_num)),
51                     label=str((number_missionaries, number_cannibals, side)))
52     graph.add_node(v)
53
54     edge = pydot.Edge(str(Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)]),
55                       str((number_missionaries, number_cannibals, side, depth_level, node_num) ), dir='forward')
56     graph.add_edge(edge)
57 else:
58     # For start node
59     v = pydot.Node(str((number_missionaries, number_cannibals, side, depth_level, node_num)),
60                     label=str((number_missionaries, number_cannibals, side)))
61     graph.add_node(v)
62     return u, v
63
64 def is_start_state(number_missionaries, number_cannibals, side):
65     return (number_missionaries, number_cannibals, side) == (3, 3, 1)
66
67 def is_goal_state(number_missionaries, number_cannibals, side):
68     return (number_missionaries, number_cannibals, side) == (0, 0, 0)
69
70 def number_of_cannibals_exceeds(number_missionaries, number_cannibals):
71     number_missionaries_right = 3 - number_missionaries
72     number_cannibals_right = 3 - number_cannibals
73     return (number_missionaries > 0 and number_cannibals > number_missionaries) \
74     or (number_missionaries_right > 0 and number_cannibals_right > number_missionaries_right)
75
76 def generate():
77     global i
78     q = deque()
79     node_num = 0
80     q.append((3, 3, 1, 0, node_num))
81
82     Parent[(3, 3, 1, 0, node_num)] = None
83
84     while q:
85
86         number_missionaries, number_cannibals, side, depth_level, node_num = q.popleft()
87         # print(number_missionaries, number_cannibals)
88         # Draw Edge from u -> v
89         # Where u = Parent[v]
90         # and v = (number_missionaries, number_cannibals, side, depth_level)
91         u, v = draw_edge(number_missionaries, number_cannibals, side, depth_level, node_num)
92
93         if is_start_state(number_missionaries, number_cannibals, side):
94             v.set_style("filled")
95             v.set_fillcolor("blue")
96             v.set_fontcolor("white")
97         elif is_goal_state(number_missionaries, number_cannibals, side):
98             v.set_style("filled")
99             v.set_fillcolor("green")
100             continue
101         # return True
102         elif number_of_cannibals_exceeds(number_missionaries, number_cannibals):
103             v.set_style("filled")
104             v.set_fillcolor("red")
105             continue
106         else:
107             v.set_style("filled")
108             v.set_fillcolor("orange")
109
110         if depth_level == max_depth:
111             return True
112
113         op = -1 if side == 1 else 1
114
115         can_be_expanded = False
116
117         # i = node_num
118         for x, y in options:
119             next_m, next_c, next_s = number_missionaries + op * x, number_cannibals + op * y, int(not side)
120
121             if Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)] is None or (next_m, next_c, next_s) \
122             != Parent[(number_missionaries, number_cannibals, side, depth_level, node_num)][3:]:
123                 if is_valid_move(next_m, next_c):
124                     can_be_expanded = True
125                     i += 1
126                     q.append((next_m, next_c, next_s, depth_level + 1, i))
127                     # Keep track of parent
128                     Parent[(next_m, next_c, next_s, depth_level + 1, i)] = \
129                     (number_missionaries, number_cannibals, side, depth_level, node_num)
130             if not can_be_expanded:
131                 v.set_style("filled")
132                 v.set_fillcolor("gray")
133         return False
134
135 if __name__ == "__main__":
136     if generate():
137         write_image()

```

```

6 import os
7 import emoji
8 import pydot
9 import random
10 from collections import deque
11
12 # Set it to bin folder of graphviz
13 os.environ["PATH"] += os.pathsep + 'C:/Program Files/Graphviz/bin'
14
15 # Dictionaries to backtrack solution nodes
16 # Parent stores parent of (m, c, s)
17 # Move stores (x, y, side) i.e number of missionaries,
18 # cannibals to be moved from left to right or right to left for particular state
19 # node_list stores pydot.Node object for particular state (m, c, s) so that we can color the solution nodes
20 Parent, Move, node_list = dict(), dict(), dict()
21
22 class Solution():
23
24     def __init__(self):
25         # Start state (3M, 3C, Left)
26         # Goal State (0M, 0C, Right)
27         # Each state gives the number of missionaries and cannibals on the left side
28
29         self.start_state = (3, 3, 1)
30         self.goal_state = (0, 0, 0)
31         self.options = [(1, 0), (0, 1), (1, 1), (0, 2), (2, 0)]
32
33         self.boat_side = ["right", "left"]
34
35         self.graph = pydot.Dot(graph_type='graph', bgcolor="#ffff99",
36                                label="fig: Missionaries and Cannibal State Space Tree", fontcolor="red", fontsize="24")
37         self.visited = {}
38         self.solved = False
39
40     def is_valid_move(self, number_missionaries, number_cannibals):
41         """
42         Checks if number constraints are satisfied
43         """
44         return (0 <- number_missionaries <- 3) and (0 <- number_cannibals <- 3)
45
46     def is_goal_state(self, number_missionaries, number_cannibals, side):
47         return (number_missionaries, number_cannibals, side) == self.goal_state
48
49     def is_start_state(self, number_missionaries, number_cannibals, side):
50         return (number_missionaries, number_cannibals, side) == self.start_state
51
52     def number_of_cannibals_exceeds(self, number_missionaries, number_cannibals):
53         number_missionaries_right = 3 - number_missionaries
54         number_cannibals_right = 3 - number_cannibals
55         return (number_missionaries > 0 and number_cannibals > number_missionaries) \
56             or (number_missionaries_right > 0 and number_cannibals_right > number_missionaries_right)
57
58     def write_image(self, file_name="state_space.png"):
59         try:
60             self.graph.write_png(file_name)
61         except Exception as e:
62             print("Error while writing file", e)
63             print(f"File {file_name} successfully written.")
64
65     def solve(self, solve_method="dfs"):
66         self.visited = dict()
67         Parent[self.start_state] = None
68         Move[self.start_state] = None
69         node_list[self.start_state] = None
70
71         return self.dfs(*self.start_state, 0) if solve_method == "dfs" else self.bfs()
72
73     def draw_legend(self):
74         """
75         Utility method to draw legend on graph if legend flag is ON
76         """
77         graphlegend = pydot.Cluster(graph_name="legend", label="Legend", fontsize="20", color="gold",
78                                    fontcolor="blue", style="filled", fillcolor="#f4f4f4")
79
80         node1 = pydot.Node("1", style="filled", fillcolor="blue", label="Start Node", fontcolor="white", width="2", fixedsize="true")
81         graphlegend.add_node(node1)
82
83         node2 = pydot.Node("2", style="filled", fillcolor="red", label="Killed Node", fontcolor="black", width="2", fixedsize="true")
84         graphlegend.add_node(node2)
85
86         node3 = pydot.Node("3", style="filled", fillcolor="yellow", label="Solution nodes", width="2", fixedsize="true")
87         graphlegend.add_node(node3)
88
89         node4 = pydot.Node("4", style="filled", fillcolor="gray", label="Can't be expanded", width="2", fixedsize="true")
90         graphlegend.add_node(node4)
91
92         node5 = pydot.Node("5", style="filled", fillcolor="green", label="Goal node", width="2", fixedsize="true")
93

```

```

94 graphlegend.add_node(node5)
95
96 node7 = pydot.Node("7", style="filled", fillcolor="gold", label="Node with child", width="2", fixedsize="true")
97 graphlegend.add_node(node7)
98
99 description = "Each node (m, c, s) represents a \nstate where 'm' is the number of\nmissionaries,\n'n' the cannibals \
100 | and '\n's' the side of the boat\n"
101 | " where '1' represents the left \nside and '0' the right side \n\nOur objective is to reach goal state (0, 0, 0)\
102 | \nfrom start state (3, 3, 1) by some \noperators = [(0, 1), (0, 2), (1, 0), (1, 1), (2, 0),]\n\n"
103 | "each tuples (x, y) inside operators \nrepresents the number of missionaries and\
104 | \ncannibals to be moved from left to right \nif c == 1 and viceversa"
105
106 node6 = pydot.Node("6", style="filled", fillcolor="gold", label= description, shape="plaintext", fontsize="20", fontcolor="red")
107 graphlegend.add_node(node6)
108
109 self.graph.add_subgraph(graphlegend)
110
111 self.graph.add_edge(pydot.Edge(node1, node2, style="invis"))
112 self.graph.add_edge(pydot.Edge(node2, node3, style="invis"))
113 self.graph.add_edge(pydot.Edge(node3, node4, style="invis"))
114 self.graph.add_edge(pydot.Edge(node4, node5, style="invis"))
115 self.graph.add_edge(pydot.Edge(node5, node7, style="invis"))
116 self.graph.add_edge(pydot.Edge(node7, node6, style="invis"))
117
118 def draw(self, *, number_missionaries_left, number_cannibals_left, number_missionaries_right, number_cannibals_right):
119     """
120     Draw state on console using emojis
121     """
122     left_m = emoji.emojize(f":old_man: " * number_missionaries_left)
123     left_c = emoji.emojize(f":ogre: " * number_cannibals_left)
124     right_m = emoji.emojize(f":old_man: " * number_missionaries_right)
125     right_c = emoji.emojize(f":ogre: " * number_cannibals_right)
126
127     print('({}{}){}{}\n'.format(left_m, left_c + " " * (14 - len(left_m) - len(left_c)), \
128     | " " * 40, " " * (12 - len(right_m) - len(right_c)) + right_m, right_c))
129     print("")
130
131 def show_solution(self):
132     # Recursively start from Goal State
133     # And find parent until start state is reached
134
135     state = self.goal_state
136     path, steps, nodes = [], [], []
137
138     while state is not None:
139         path.append(state)
140         steps.append(Move[state])
141         nodes.append(node_list[state])
142
143         state = Parent[state]
144
145     steps, nodes = steps[::-1], nodes[::-1]
146
147     number_missionaries_left, number_cannibals_left = 3, 3
148     number_missionaries_right, number_cannibals_right = 0, 0
149
150     print("\n" * 60)
151     self.draw(number_missionaries_left-number_missionaries_left, number_cannibals_left-number_cannibals_left,
152     | number_missionaries_right-number_missionaries_right, number_cannibals_right-number_cannibals_right)
153
154     for i, ((number_missionaries, number_cannibals, side), node) in enumerate(zip(steps[1:], nodes[1:])):
155
156         if node.get_label() != str(self.start_state):
157             node.set_style("filled")
158             node.set_fillcolor("yellow")
159
160         print(f"Step {i + 1}: Move {number_missionaries} missionaries and {number_cannibals} \
161         | cannibals from {self.boat_side[side]} to {self.boat_side[int(not side)]}.")
162
163         op = -1 if side == 1 else 1
164
165         number_missionaries_left = number_missionaries_left + op * number_missionaries
166         number_cannibals_left = number_cannibals_left + op * number_cannibals
167
168         number_missionaries_right = number_missionaries_right - op * number_missionaries
169         number_cannibals_right = number_cannibals_right - op * number_cannibals
170
171         self.draw(number_missionaries_left-number_missionaries_left, number_cannibals_left-number_cannibals_left,
172         | number_missionaries_right-number_missionaries_right, number_cannibals_right-number_cannibals_right)
173
174         print("Congratulations!!! you have solved the problem")
175         print("\n" * 60)
176
177 def draw_edge(self, number_missionaries, number_cannibals, side, depth_level):
178     u, v = None, None
179     if Parent[(number_missionaries, number_cannibals, side)] is not None:
180         u = pydot.Node(str(Parent[(number_missionaries, number_cannibals, side)] + (depth_level - 1,)),

```



```

181         label=str(Parent[((number_missionaries, number_cannibals, side))]))
182     self.graph.add_node(u)
183
184     v = pydot.Node(str((number_missionaries, number_cannibals, side, depth_level)),
185                   label=str((number_missionaries, number_cannibals, side)))
186     self.graph.add_node(v)
187
188     edge = pydot.Edge(str(Parent[(number_missionaries, number_cannibals, side)] + (depth_level - 1, )),
189                      str((number_missionaries, number_cannibals, side, depth_level)), dir='forward')
190     self.graph.add_edge(edge)
191
192     else:
193         # For start node
194         v = pydot.Node(str((number_missionaries, number_cannibals, side, depth_level)),
195                       label=str((number_missionaries, number_cannibals, side)))
196         self.graph.add_node(v)
197     return u, v
198
199 def bfs(self):
200     q = deque()
201     q.append(self.start_state + (0, ))
202     self.visited[self.start_state] = True
203
204     while q:
205         number_missionaries, number_cannibals, side, depth_level = q.popleft()
206         # Draw Edge from u -> v
207         # Where u = Parent[v]
208         # and v = (number_missionaries, number_cannibals, side, depth_level)
209         u, v = self.draw_edge(number_missionaries, number_cannibals, side, depth_level)
210
211         if self.is_start_state(number_missionaries, number_cannibals, side):
212             v.set_style("filled")
213             v.set_fillcolor("blue")
214             v.set_fontcolor("white")
215         elif self.is_goal_state(number_missionaries, number_cannibals, side):
216             v.set_style("filled")
217             v.set_fillcolor("green")
218             return True
219         elif self.number_of_cannibals_exceeds(number_missionaries, number_cannibals):
220             v.set_style("filled")
221             v.set_fillcolor("red")
222             continue
223         else:
224             v.set_style("filled")
225             v.set_fillcolor("orange")
226
227         op = -1 if side == 1 else 1
228
229         can_be_expanded = False
230
231         for x, y in self.options:
232             next_m, next_c, next_s = number_missionaries + op * x, number_cannibals + op * y, int(not side)
233             if (next_m, next_c, next_s) not in self.visited:
234                 if self.is_valid_move(next_m, next_c):
235                     can_be_expanded = True
236                     self.visited[(next_m, next_c, next_s)] = True
237                     q.append((next_m, next_c, next_s, depth_level + 1))
238
239                     # Keep track of parent and corresponding move
240                     Parent[(next_m, next_c, next_s)] = (number_missionaries, number_cannibals, side)
241                     Move[(next_m, next_c, next_s)] = (x, y, side)
242                     node_list[(next_m, next_c, next_s)] = v
243
244             if not can_be_expanded:
245                 v.set_style("filled")
246                 v.set_fillcolor("gray")
247         return False
248
249 def dfs(self, number_missionaries, number_cannibals, side, depth_level):
250     self.visited[(number_missionaries, number_cannibals, side)] = True
251
252     # Draw Edge from u -> v
253     # Where u = Parent[v]
254     u, v = self.draw_edge(number_missionaries, number_cannibals, side, depth_level)
255
256
257     if self.is_start_state(number_missionaries, number_cannibals, side):
258         v.set_style("filled")
259         v.set_fillcolor("blue")
260     elif self.is_goal_state(number_missionaries, number_cannibals, side):
261         v.set_style("filled")
262         v.set_fillcolor("green")
263         return True
264     elif self.number_of_cannibals_exceeds(number_missionaries, number_cannibals):
265         v.set_style("filled")
266         v.set_fillcolor("red")
267         return False
268     else:

```



```

269         v.set_style("filled")
270         v.set_fillcolor("orange")
271
272     solution_found = False
273     operation = -1 if side == 1 else 1
274
275     can_be_expanded = False
276
277     for x, y in self.options:
278         next_m, next_c, next_s = number_missionaries + operation * x, number_cannibals + operation * y, int(not side)
279
280         if (next_m, next_c, next_s) not in self.visited:
281             if self.is_valid_move(next_m, next_c):
282                 can_be_expanded = True
283                 # Keep track of Parent state and corresponding move
284                 Parent[(next_m, next_c, next_s)] = (number_missionaries, number_cannibals, side)
285                 Move[(next_m, next_c, next_s)] = (x, y, side)
286                 node_list[(next_m, next_c, next_s)] = v
287
288                 solution_found = (solution_found or self.dfs(next_m, next_c, next_s, depth_level + 1))
289
290             if solution_found:
291                 return True
292
293         if not can_be_expanded:
294             v.set_style("filled")
295             v.set_fillcolor("gray")
296
297     self.solved = solution_found
298     return solution_found

```

```

main.py x
5  from solve import Solution
6  import argparse
7  import itertools
8
9  arg = argparse.ArgumentParser()
10 arg.add_argument("-m", "--method", required=False, help="Specify which method to use")
11 arg.add_argument("-l", "--legend", required=False, help="Specify if you want to display legend on graph")
12
13 args = vars(arg.parse_args())
14
15 solve_method = args.get("method", "bfs")
16 legend_flag = args.get("legend", False)
17
18
19 def main():
20     s = Solution()
21
22     if(s.solve(solve_method)):
23
24         # Display Solution on console
25         s.show_solution()
26
27         output_file_name = f"{solve_method}"
28         # Draw legend if legend_flag is set
29         if legend_flag:
30             if legend_flag[0].upper() == 'T' :
31                 output_file_name += "_legend.png"
32                 s.draw_legend()
33             else:
34                 output_file_name += ".png"
35         else:
36             output_file_name += ".png"
37
38         # Write State space tree
39         s.write_image(output_file_name)
40     else:
41         raise Exception("No solution found")
42
43
44 if __name__ == "__main__":
45     main()

```

- a. Cài đặt thư viện graphviz tải về từ link: <https://graphviz.org/download/> và đặt đường dẫn tới thư mục bin của graphviz đã cài đặt trên máy tính.

```
generate_full_space_tree.py
10
11 # Set it to bin folder of graphviz
12 os.environ["PATH"] += os.pathsep + 'C:/Program Files/Graphviz/bin'
```

```
solve.py
11
12 # Set it to bin folder of graphviz
13 os.environ["PATH"] += os.pathsep + 'C:/Program Files/Graphviz/bin'
```

b. Cài đặt các yêu cầu trong file “requirements.txt”:

```
requirements.txt - Notepad
File Edit Format View Help
emoji==0.5.4
pydot==1.4.1
pyparsing==2.4.5
```

`pip install -r requirements.txt`

```
C:\WINDOWS\system32\cmd.exe
C:\Users\Huynh>cd C:\Users\Huynh\Desktop\Tuan2
C:\Users\Huynh\Desktop\Tuan2>pip install -r requirements.txt
```

c. Khởi tạo cây không gian trạng thái:

`python generate_full_space_tree.py -d 8` (với d là độ sâu (depth=8))

```
C:\Users\Huynh\Desktop\Tuan2>python generate_full_space_tree.py -d 8
File state_space_8.png successfully written.
```

`python generate_full_space_tree.py -d 20` (depth = 20)

```
C:\Users\Huynh\Desktop\Tuan2>python generate_full_space_tree.py -d 20
File state_space_20.png successfully written.
```

Làm tương tự với depth = 10, depth = 40, ...

d. Cây DFS:

- DFS:

`python main.py -m dfs`

- DFS với legend:

python main.py -m dfs -l True

```
C:\Users\Huynh\Desktop\Tuan2>python main.py -m dfs -l True
*****
[ ] [ ] [ ] [ ] [ ] [ ] _____

Step 1: Move 1 missionaries and 1 cannibals from left to right.
[ ] [ ] [ ] [ ] _____ [ ] [ ]

Step 2: Move 1 missionaries and 0 cannibals from right to left.
[ ] [ ] [ ] [ ] [ ] _____ [ ]

Step 3: Move 0 missionaries and 2 cannibals from left to right.
[ ] [ ] [ ] _____ [ ] [ ] [ ]

Step 4: Move 0 missionaries and 1 cannibals from right to left.
[ ] [ ] [ ] [ ] _____ [ ] [ ]

Step 5: Move 2 missionaries and 0 cannibals from left to right.
[ ] [ ] _____ [ ] [ ] [ ] [ ]

Step 6: Move 1 missionaries and 1 cannibals from right to left.
[ ] [ ] [ ] [ ] _____ [ ] [ ]

Step 7: Move 2 missionaries and 0 cannibals from left to right.
[ ] [ ] _____ [ ] [ ] [ ] [ ]

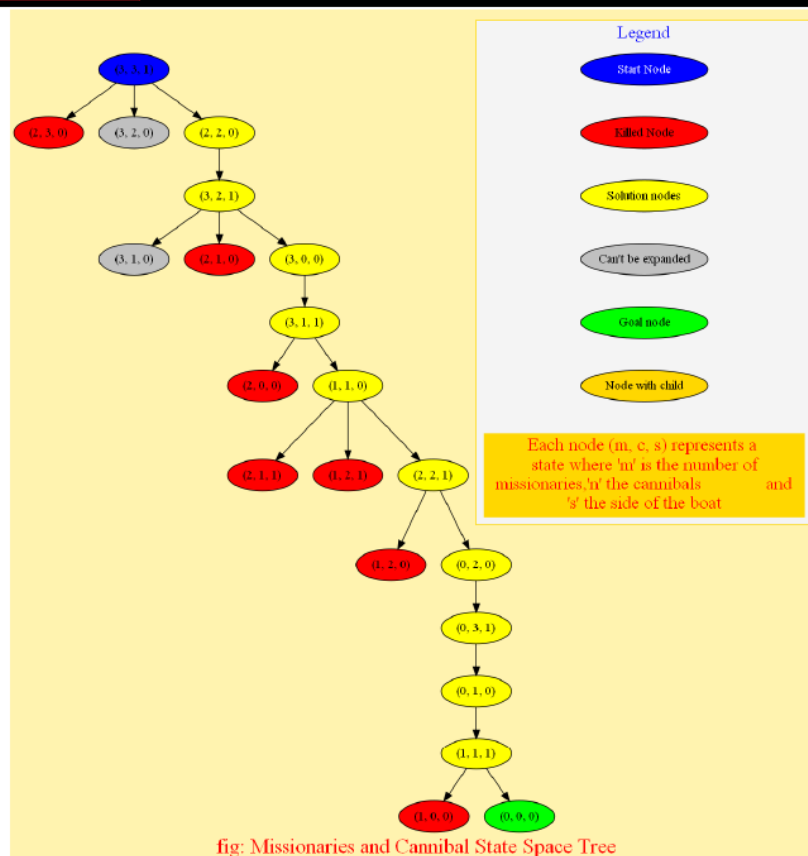
Step 8: Move 0 missionaries and 1 cannibals from right to left.
[ ] [ ] [ ] _____ [ ] [ ] [ ]

Step 9: Move 0 missionaries and 2 cannibals from left to right.
[ ] _____ [ ] [ ] [ ] [ ] [ ]

Step 10: Move 1 missionaries and 0 cannibals from right to left.
[ ] [ ] _____ [ ] [ ] [ ] [ ]

Step 11: Move 1 missionaries and 1 cannibals from left to right.
[ ] [ ] _____ [ ] [ ] [ ] [ ] [ ]

Congratulations!!! you have solved the problem
*****
File dfs_legend.png successfully written.
```



e. Cây BFS:

- BFS:

python main.py -m bfs

```
C:\Users\Huynh\Desktop\Tuan2>python main.py -m bfs
*****
0 0 0 0 0 0
Step 1: Move 1 missionaries and 1 cannibals from left to right.
0 0 0 0
Step 2: Move 1 missionaries and 0 cannibals from right to left.
0 0 0 0 0
Step 3: Move 0 missionaries and 2 cannibals from left to right.
0 0 0
Step 4: Move 0 missionaries and 1 cannibals from right to left.
0 0 0 0
Step 5: Move 2 missionaries and 0 cannibals from left to right.
0 0
Step 6: Move 1 missionaries and 1 cannibals from right to left.
0 0 0 0
Step 7: Move 2 missionaries and 0 cannibals from left to right.
0 0
Step 8: Move 0 missionaries and 1 cannibals from right to left.
0 0 0
Step 9: Move 0 missionaries and 2 cannibals from left to right.
0
Step 10: Move 1 missionaries and 0 cannibals from right to left.
0 0
Step 11: Move 1 missionaries and 1 cannibals from left to right.
0 0 0 0 0 0
Congratulations!!! you have solved the problem
*****
File bfs.png successfully written.
```

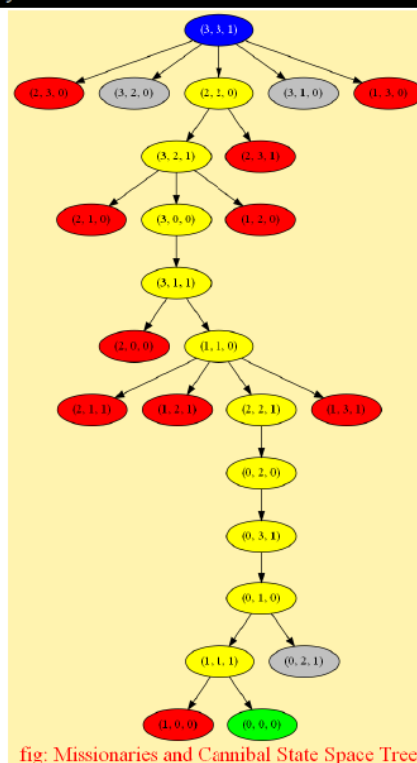


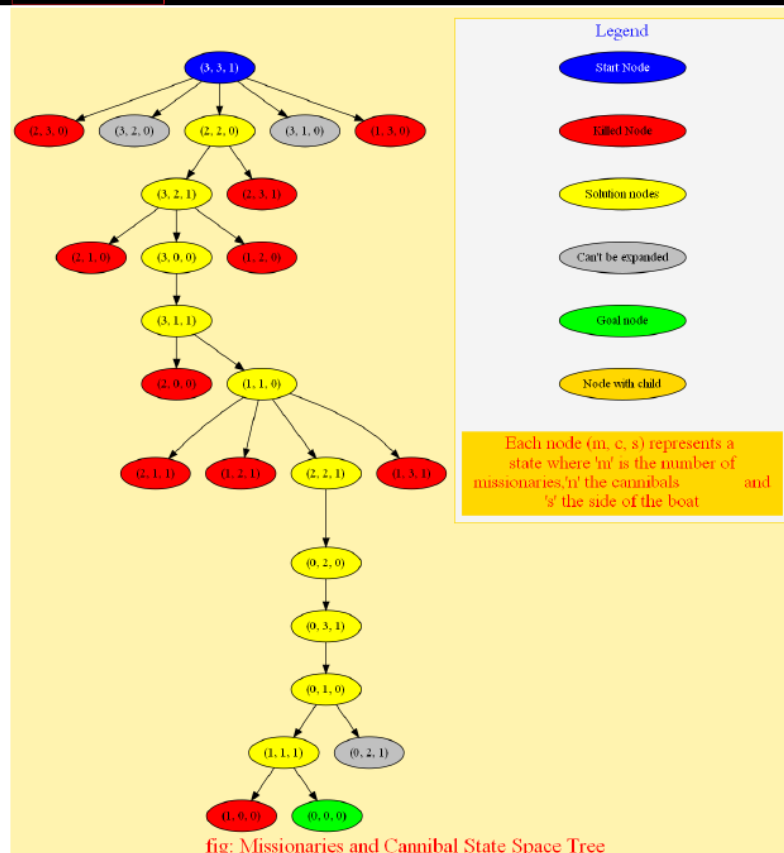
fig: Missionaries and Cannibal State Space Tree

- BFS với legend:

python main.py -m bfs -l True

```
C:\Users\Huynh\Desktop\Tuan2>python main.py -m bfs -l True
*****
[ ] [ ] [ ] [ ] [ ] [ ]
Step 1: Move 1 missionaries and 1 cannibals from left to right.
[ ] [ ] [ ] [ ] [ ] [ ]
Step 2: Move 1 missionaries and 0 cannibals from right to left.
[ ] [ ] [ ] [ ] [ ] [ ]
Step 3: Move 0 missionaries and 2 cannibals from left to right.
[ ] [ ] [ ] [ ] [ ] [ ]
Step 4: Move 0 missionaries and 1 cannibals from right to left.
[ ] [ ] [ ] [ ] [ ] [ ]
Step 5: Move 2 missionaries and 0 cannibals from left to right.
[ ] [ ] [ ] [ ] [ ] [ ]
Step 6: Move 1 missionaries and 1 cannibals from right to left.
[ ] [ ] [ ] [ ] [ ] [ ]
Step 7: Move 2 missionaries and 0 cannibals from left to right.
[ ] [ ] [ ] [ ] [ ] [ ]
Step 8: Move 0 missionaries and 1 cannibals from right to left.
[ ] [ ] [ ] [ ] [ ] [ ]
Step 9: Move 0 missionaries and 2 cannibals from left to right.
[ ] [ ] [ ] [ ] [ ] [ ]
Step 10: Move 1 missionaries and 0 cannibals from right to left.
[ ] [ ] [ ] [ ] [ ] [ ]
Step 11: Move 1 missionaries and 1 cannibals from left to right.
[ ] [ ] [ ] [ ] [ ] [ ]

Congratulations!!! you have solved the problem
*****
File bfs_legend.png successfully written.
```



4. Yêu cầu:

- Cài đặt và thực thi chương trình.
- Viết báo cáo trình bày:
 - ✿ Nếu chương trình bị báo lỗi thì lỗi ở dòng nào và sửa lại như thế nào? (Nếu có).
 - ✿ Đọc hiểu code đã cho và trình bày lại chi tiết hơn đoạn code cho sẵn dùng để làm gì?